

Robotic Navigation and Exploration

HW3: Deep Reinforcement Learning

Min-Chun Hu anitahu@cs.nthu.edu.tw
CS, NTHU

HW3 Overview

1. HW3-1: Deep Reinforcement Learning on Path Tracking (40%)

- Observation/action/reward fixed → *complete PPO/agent implementation*

2. HW3-2: Deep Reinforcement Learning on Proly (50%)

- Algorithm/Observation/Action fixed → *complete reward design*

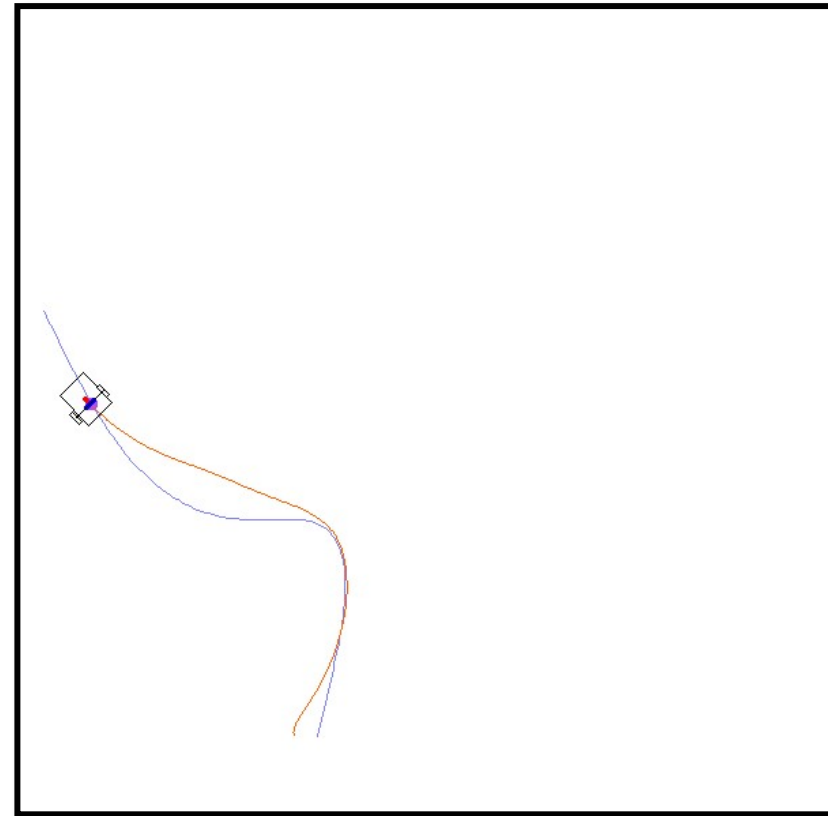
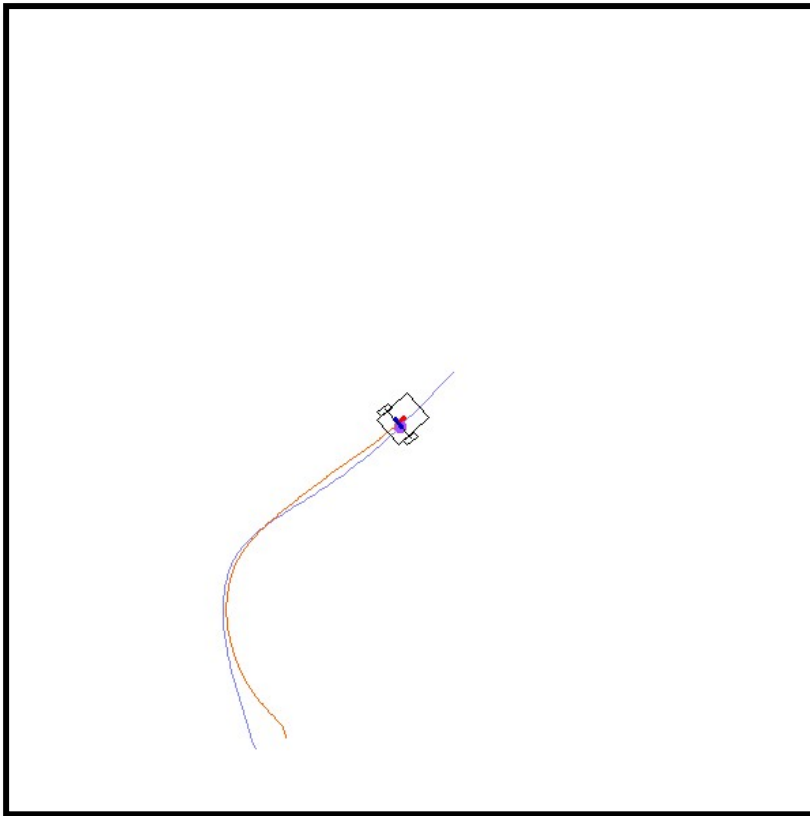
3. Report (10%)

- Write two reports for HW3-1 and HW3-2, 5% each
- Briefly explain your implementation, writing a lot won't get you more points!

HW3-1: Path Tracking

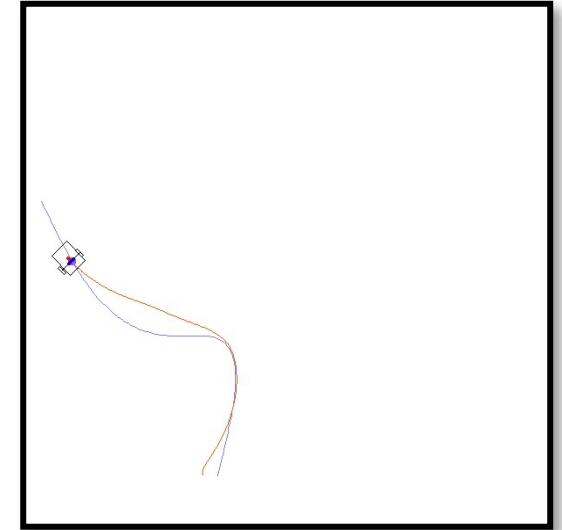
PathTracking Environment

- Controls the car to follow the given path



PathTracking Environment

- **Observation Space:** $o \in \mathbb{R}^{14}$
 - **Elements 0-2:** Car pose from 2 timestep ago (x, y, yaw)
 - **Elements 3-5:** Car pose from 1 timestep ago (x, y, yaw)
 - **Elements 6-7:** Coordinate of the nearest point on path (x, y)
 - **Elements 8-13:** Coordinates of the next 3 waypoint on path (x, y)
 - where $x, y \in [0, 1]$ and $\text{yaw} \in [0, 2\pi]$
- **Action Space:** Angular velocity $a \in [-1, 1]$
- **Reward:** $r = 0.8 r_d + 0.2 r_y + r_p$
 - $r_d = \exp(-0.1|p - p'|^2)$: Distance reward
 - $r_y = \exp(-0.1|\theta - \theta'|^2)$: Yaw reward
 - $r_p = \begin{cases} 0.1 & , \text{if progress is positive} \\ 0 & , \text{if progress is 0} \\ -1 & , \text{if progress is negative} \end{cases}$



- **Termination:** 100% progress or reach 400 timesteps

Note:

The value range of action is normalized to $[-1, 1]$. If the input action value is out of range, the value will be clipped.

PPO Algorithm

Algorithm 1 PPO, Actor-Critic Style

```
for iteration=1,2,... do
  for actor=1,2,...,  $N$  do
    Run policy  $\pi_{\theta_{\text{old}}}$  in environment for  $T$  timesteps
    Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$ 
  end for
  Optimize surrogate  $L$  wrt  $\theta$ , with  $K$  epochs and minibatch size  $M \leq NT$ 
   $\theta_{\text{old}} \leftarrow \theta$ 
end for
```

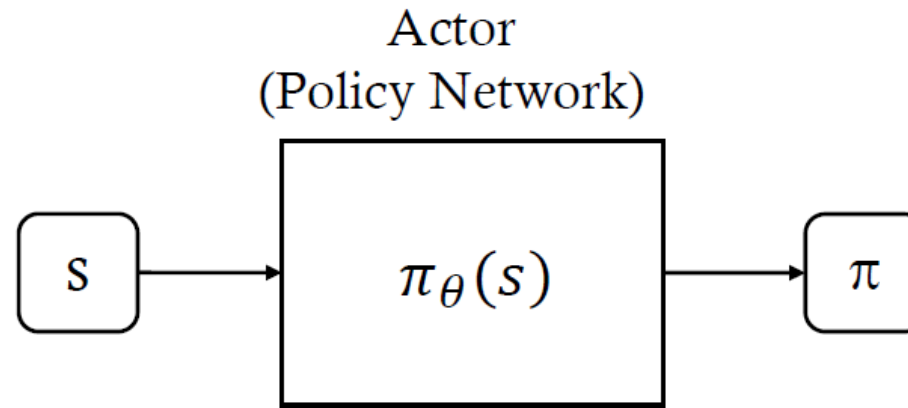
Run an episode to
collect data

Train the model using
the collected data

PPO Algorithm

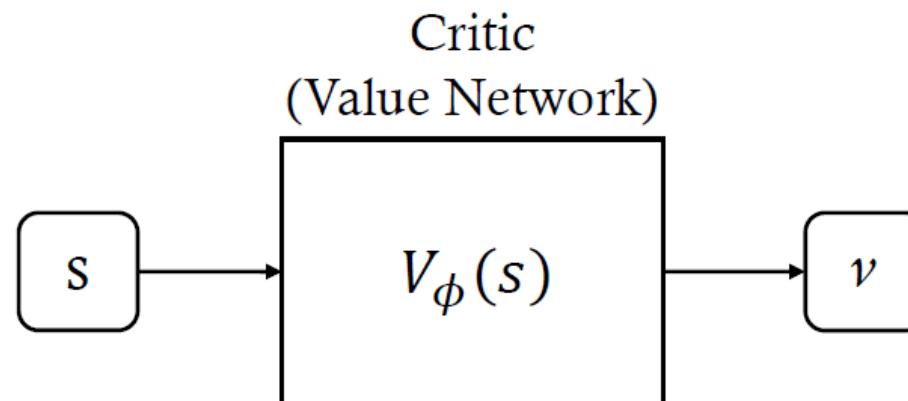
- Actor Network:

$$\pi_{\theta}(a|s)$$



- Critic Network:

$$V_{\omega}(s)$$



PPO Algorithm

- Value Loss:

$$L(\omega) = \frac{1}{2} \sum_n [G_t - V_\omega(s_t^{(n)})]^2$$

- Policy Gradient Loss:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

$$L^{CLIP}(\theta) = E_{(s_t, a_t) \sim \pi_{\theta_{old}}} \left[\min \{ \underline{r_t(\theta) A^{\theta_{old}}(s_t, a_t)}, \quad \underline{\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A^{\theta_{old}}(s_t, a_t)} \} \right]$$

Repo Structure

- HW3-1/

- **PathTracking/** *# PathTracking Utilities*
- **Simulation/** *# Simulation framework*
- **model.py**
 - Define policy & value network architecture **(TODO 1 & 2)**
- **env_runner.py**
 - Run simulation environment to collect batched data **(TODO 3)**
- **agent.py**
 - PPO algorithm implementation **(TODO 4)**
- **train.py** *# Training script (TODO 5)*
- **play.py** *# Inference script*
- **eval.py** *# Evaluation script*
- **plot.py** *# Training curve visualization*

Environment & other utilities

- **cubic_spline.py** & **multi_env.py** & **wrapper.py**

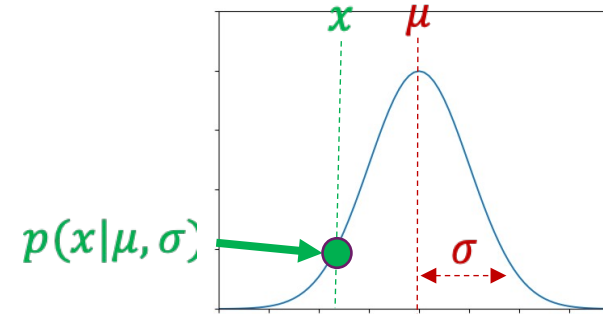
(model.py) Diagonal Gaussian Distribution

1. FixedNormal

```
# Normal distribution module with fixed mean and std.
class FixedNormal(torch.distributions.Normal):
    # Log-probability
    def log_probs(self, actions) → Any:
        return super().log_prob(actions).sum(-1)

    # Entropy
    def entropy(self) → Any:
        return super().entropy().sum(-1)

    # Mode
    def mode(self) → Any:
        return self.mean
```



$$N(x|\mu, \Sigma) = \frac{1}{(2\pi)^{(D/2)}} \frac{1}{|\Sigma|^{1/2}} \exp\left\{-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right\}$$

$$\ln p(X|\mu, \Sigma) = -\frac{ND}{2} \ln(2\pi) - \frac{N}{2} \ln |\Sigma| - \frac{1}{2} \sum_{n=1}^N (x_n - \mu)^T \Sigma^{-1} (x_n - \mu)$$

(model.py) Diagonal Gaussian Distribution

2. DiagGaussian

```
# Diagonal Gaussian distribution
class DiagGaussian(nn.Module):
    # Constructor
    def __init__(self, inp_dim, out_dim, std=0.5):
        super().__init__()

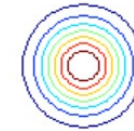
        def init(m):
            return init(m, nn.init.orthogonal_, lambda x: nn.init.constant_(x, 0))

        self.fc_mean = init(nn.Linear(inp_dim, out_dim))
        self.std = torch.full((out_dim,), std)

    # Forward
    def forward(self, x):
        mean = self.fc_mean(x)
        return FixedNormal(mean, self.std.to(x.device))
```

$$\text{mean} = \begin{bmatrix} m_1 \\ m_2 \\ \vdots \end{bmatrix}, \quad \text{std} = \begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \vdots \end{bmatrix} \Rightarrow \Sigma = \begin{bmatrix} \sigma_1^2 & 0 & \dots & 0 \\ 0 & \sigma_2^2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \sigma_k^2 \end{bmatrix}$$

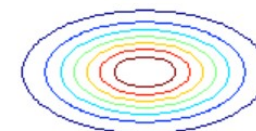
sigma = [1 0; 0 1]



sigma = [1 0; 0 4]



sigma = [4 0; 0 1]



(model.py) Policy Network

- **PolicyNet** class contains the following methods:
 - **`__init__(self, s_dim, a_dim, std)`**
 - (TODO) Initializes the policy network with state/action dimensions and configures network layers.
 - **`forward(self, state, deterministic)`**
 - Computes action and log-probability from state, sampling stochastically or deterministically based on input flag.
 - **`action_step(self, state, deterministic)`**
 - Outputs a single action from state, with optional deterministic mode for evaluation.
 - **`evaluate(self, state, action)`**
 - Computes log-probabilities and entropy for given state-action pairs, used for policy gradient calculations.

TODO 1 (model.py)

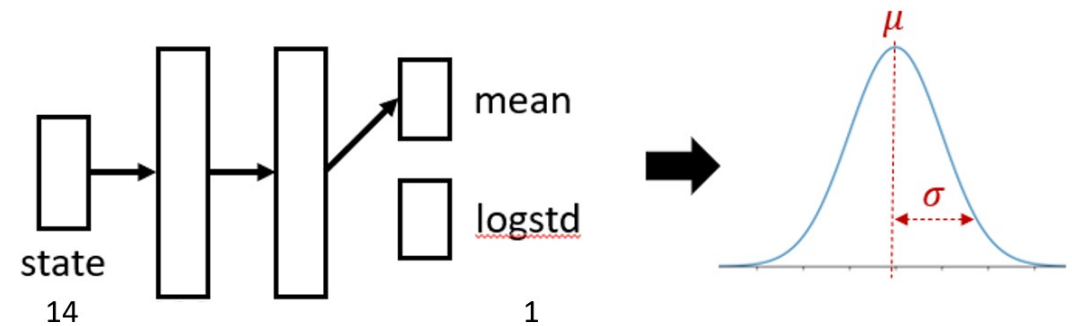
- Implement the policy network architecture

```
# Policy network
class PolicyNet(nn.Module):
    # Constructor
    def __init__(self, s_dim, a_dim, std=0.5):
        super().__init__()

        def init(m):
            return init(
                m,
                nn.init.orthogonal_,
                lambda x: nn.init.constant_(x, 0),
                nn.init.calculate_gain("relu"),
            )

        # TODO 1: policy network architecture
        """
        self.main = ...
        self.dist = ...
        """
```

(Hint: use `nn.Sequential` & `DiagGaussian`)



(model.py) Value Network

- **ValueNet** class contains the following methods:
 - **`__init__(self, s_dim)`**
 - (TODO) Initializes the value network with state dimension and configures network layers.
 - **`forward(self, state)`**
 - Computes the state value estimate by passing state through the network and extracting the scalar output.

TODO 2 (model.py)

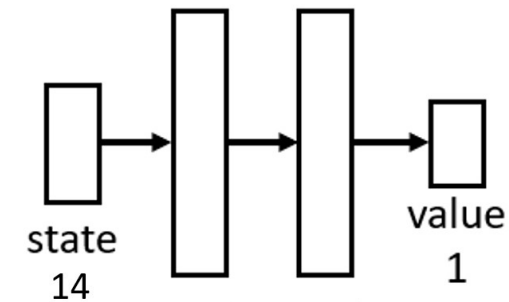
- Implement the value network architecture

```
# Value network
class ValueNet(nn.Module):
    # Constructor
    def __init__(self, s_dim):
        super().__init__()

        def init_(m):
            return init(
                m,
                nn.init.orthogonal_,
                lambda x: nn.init.constant_(x, 0),
                nn.init.calculate_gain("relu"),
            )

        # TODO 2: value network architecture
        """
        self.main = ...
        """
```

(Hint: use `nn.Sequential`)



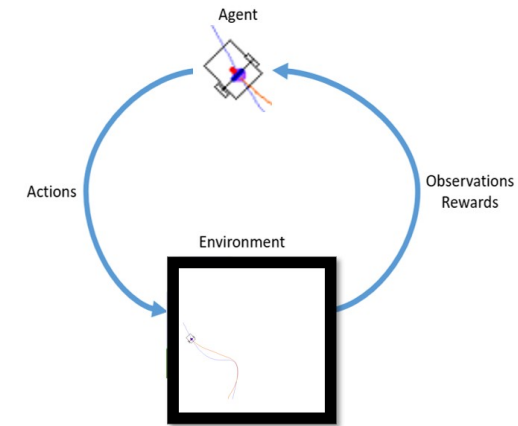
(env_runner.py) EnvRunner

- **EnvRunner** class contains the following methods:
 - **__init__(self, env, s_dim, a_dim, n_step, gamma, lamb, device)**
 - Initializes the environment runner with environment, network dimensions, and pre-allocates storage buffers for trajectory data collection.
 - **run(self, policy_net, value_net)**
 - (TODO) Collects n-step trajectories from parallel environments and returns batches of states, actions, log-probabilities, values, and GAE returns for training.
 - **record(self)**
 - Tracks cumulative rewards and episode lengths, storing completed episode metrics in buffers for performance monitoring.
 - **get_performance(self)**
 - Computes and returns mean/std of episode returns and mean episode length from the recorded episode history.

TODO 3 (env_runner.py)

- Complete the **run** method

```
# Run n steps to get a batch
def run(self, policy_net, value_net):
    # 1. Run n steps
    # -----
    for step in range(self.n_step):
        # self.states : (n_env, s_dim)
        # actions      : (n_env, a_dim)
        # self.dones   : (n_env)
        # a_logps      : (n_env)
        # values       : (n_env)
        # rewards      : (n_env)
        # TODO 3: Run a step to collect data
        """
        self.mb_states[step, :] = ...
        self.mb_dones[step, :] = ...
        self.mb_actions[step, :] = ...
        self.mb_a_logps[step, :] = ...
        self.mb_values[step, :] = ...
        self.states, rewards, self.dones, info = self.env.step(actions)
        self.mb_rewards[step, :] = ...
        """
```



(Hint: use **value_net** & **policy_net**)

```
mb_states: (n_step, n_env, s_dim)
mb_actions: (n_step, n_env, a_dim)
mb_dones: (n_step, n_env)
mb_a_logps: (n_step, n_env)
mb_values: (n_step, n_env)
mb_rewards: (n_step, n_env)
```

(agent.py) PPO

- **PPO** class contains the following methods:
 - **`__init__(self, policy_net, value_net, lr, max_grad_norm, clip_val, sample_n_epoch, sample_mb_size, mb_size, device)`**
 - Initializes the PPO agent with policy/value networks, optimizers, and hyperparameters for clipping, learning rate, and batch sampling.
 - **`train(self, mb_states, mb_actions, mb_old_values, mb_advs, mb_returns, mb_old_a_logps)`**
 - **(TODO)** Performs PPO training over multiple epochs by sampling mini-batches, computing clipped policy and value losses, and updating both networks with gradient clipping.
 - **`lr_decay(self, it, n_it)`**
 - Applies linear learning rate decay to both actor and critic optimizers.

TODO 4 (agent.py)

- Implement the policy gradient loss

```
# Train PPO
def train(self, mb_states, mb_actions, mb_old_values, mb_advs, mb_returns, mb_old_a_logps):
    mb_states = torch.from_numpy(mb_states).to(self.device)
    mb_actions = torch.from_numpy(mb_actions).to(self.device)
    mb_old_values = torch.from_numpy(mb_old_values).to(self.device)
    mb_advs = torch.from_numpy(mb_advs).to(self.device)
    mb_returns = torch.from_numpy(mb_returns).to(self.device)
    mb_old_a_logps = torch.from_numpy(mb_old_a_logps).to(self.device)

    for i in range(self.sample_n_epoch):
        np.random.shuffle(self.rand_idx)

        for j in range(self.sample_n_mb):
            sample_idx = self.rand_idx[j * self.sample_mb_size : (j + 1) * self.sample_mb_size]
            sample_states = mb_states[sample_idx]
            sample_actions = mb_actions[sample_idx]
            sample_old_values = mb_old_values[sample_idx]
            sample_advs = mb_advs[sample_idx]
            sample_returns = mb_returns[sample_idx]
            sample_old_a_logps = mb_old_a_logps[sample_idx]

            sample_a_logps, sample_ents = self.policy_net.evaluate(sample_states, sample_actions)
            sample_values = self.value_net(sample_states)

            # TODO 4: Policy gradient loss for PPO
            """
            pg_loss = ...
            """
```

(Hint: Use `sample_a_logps` & `sample_old_a_logps` to compute the probability ratio)

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$$

$$L^{CLIP}(\theta) = E_{(s_t, a_t) \sim \pi_{\theta_{old}}} \left[\min \{ r_t(\theta) A^{\theta_{old}}(s_t, a_t), \right. \\ \left. \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A^{\theta_{old}}(s_t, a_t) \} \right]$$

TODO 5 (train.py)

```
def main():  
    # TODO 5: Adjust these parameters if needed  
    # Parameters that can be modified  
    # -----  
    n_env = 8  
    n_step = 128  
    sample_mb_size = 64  
    sample_n_epoch = 4  
    a_std = 0.5  
    lamb = 0.95  
    gamma = 0.99  
    clip_val = 0.2  
    lr = 1e-4  
    n_iter = 30000  
    device = "cpu"
```

Note:

Too many environments may cause "OSError: The paging file is too small for this operation to complete". If so, please set a smaller **n_env**.

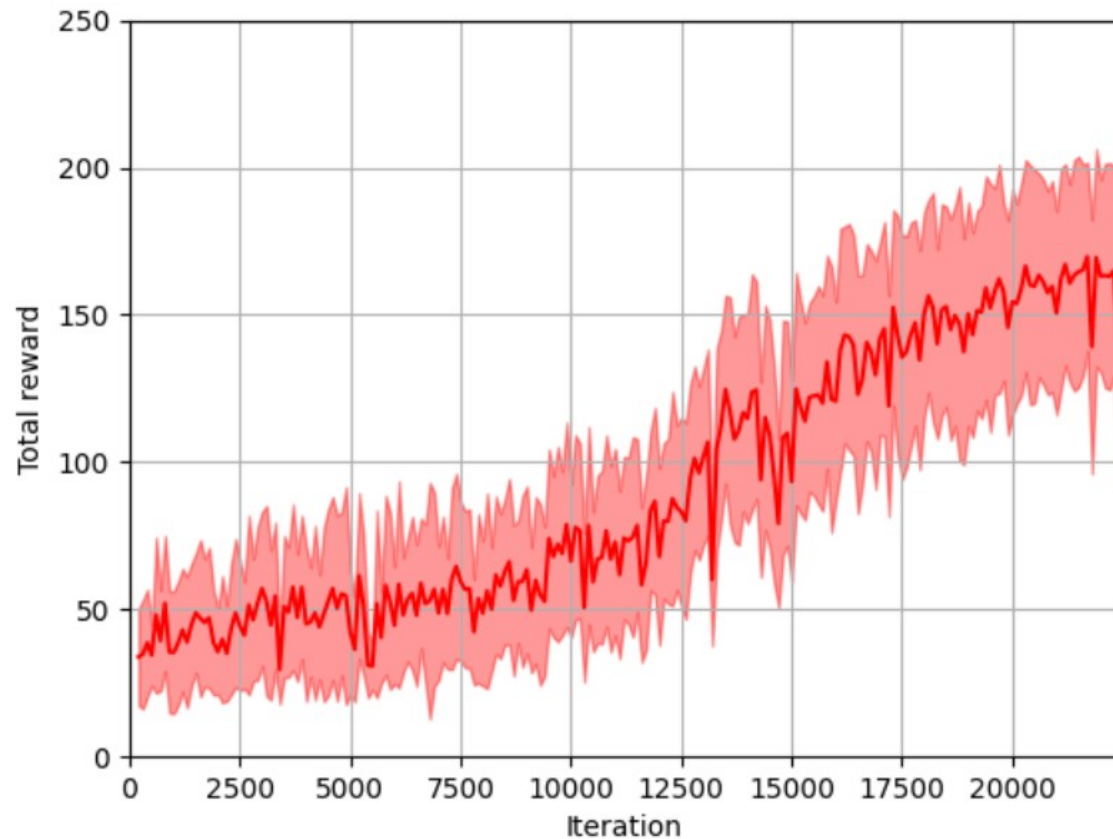
- **n_env**: number of environments (actors)
- **n_step**: number of step for runner
- **sample_mb_size**: sample mini-batch size
- **sample_n_epoch**: number of epoch in PPO
- **a_std**: std. dev. of action distribution
- **lamb**: gae factor λ
- **gamma**: discount factor γ
- **clip_val**: clip value ϵ
- **lr**: learning rate
- **n_iter**: number of iteration

Shell Commands

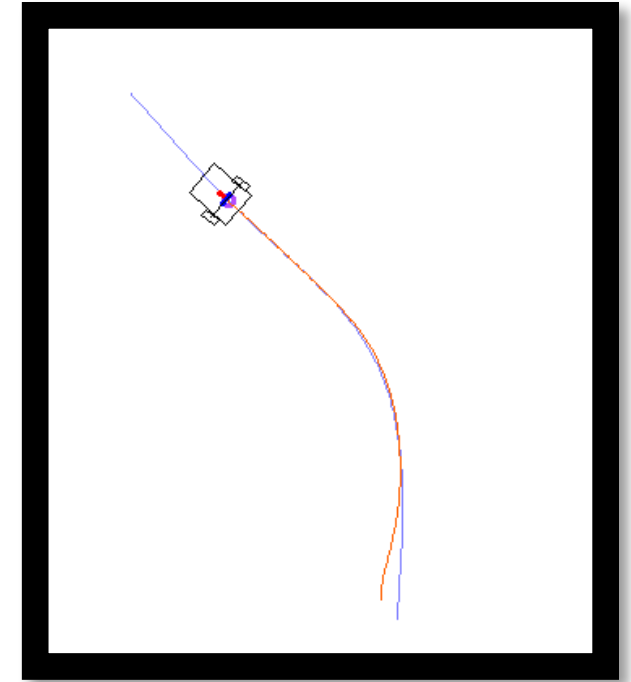
- **Training:** `python train.py`
- **Inferencing:** `python play.py`
- **Evaluation:** `python eval.py`
- **Training Visualization:** `python plot.py`

Experimental Results

plot.py



play.py



eval.py

```
Total reward = 258.102730, length = 320
Total reward = 206.884708, length = 325
Total reward = 151.192916, length = 307
Total reward = 169.869626, length = 282
Total reward = 150.867508, length = 306
Total reward = 174.895304, length = 298
Total reward = 170.725990, length = 295
Evaluation Score: 159.6902
```

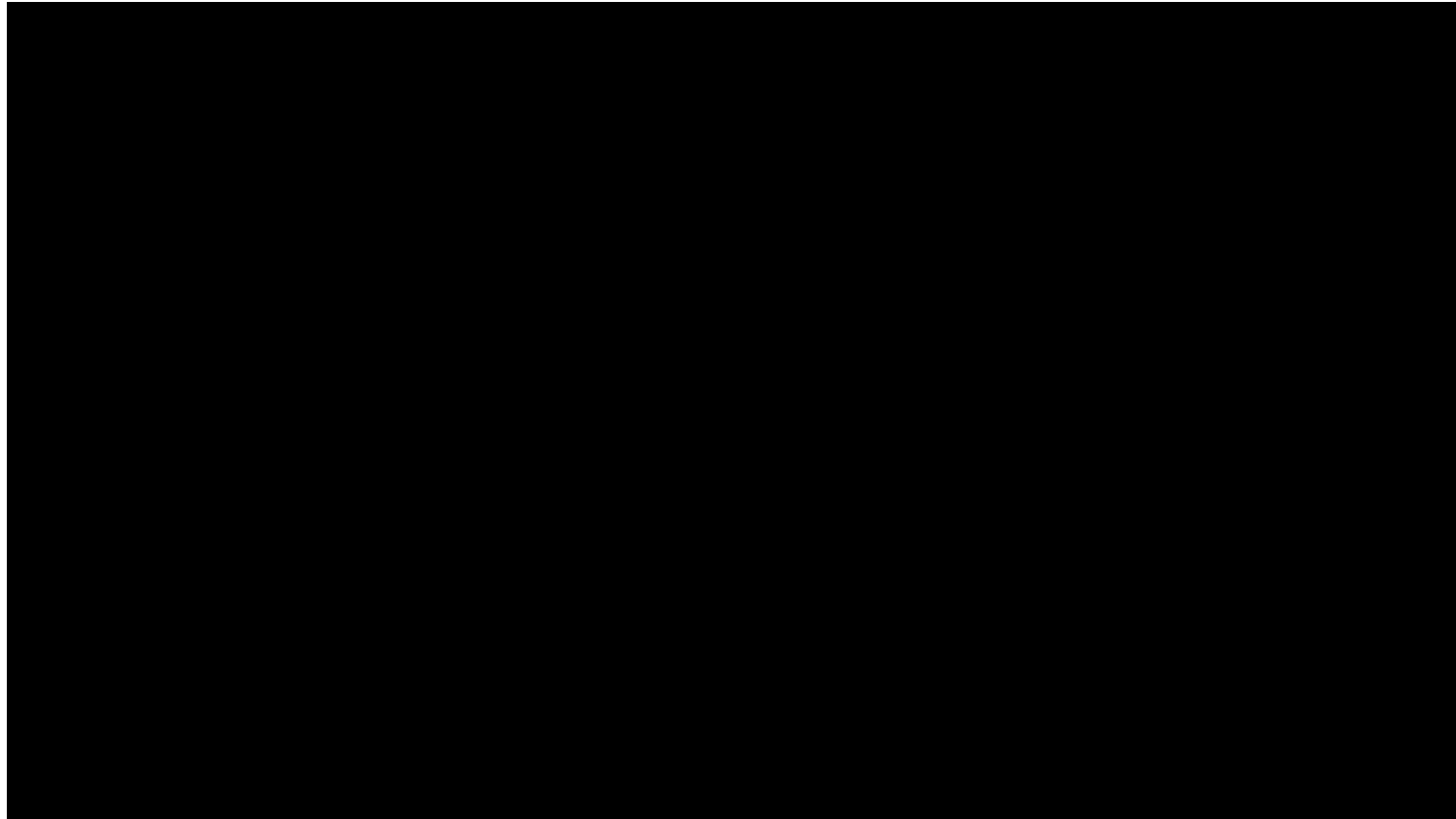
Setup Requirements

- Python version: 3.6+
- Install the following python packages with your package manager of choice
 - `numpy>=1.20,<2.0`
 - `matplotlib`
 - `opencv-python`
 - `torch`
 - `cloudpickle`

HW3-2: Proly

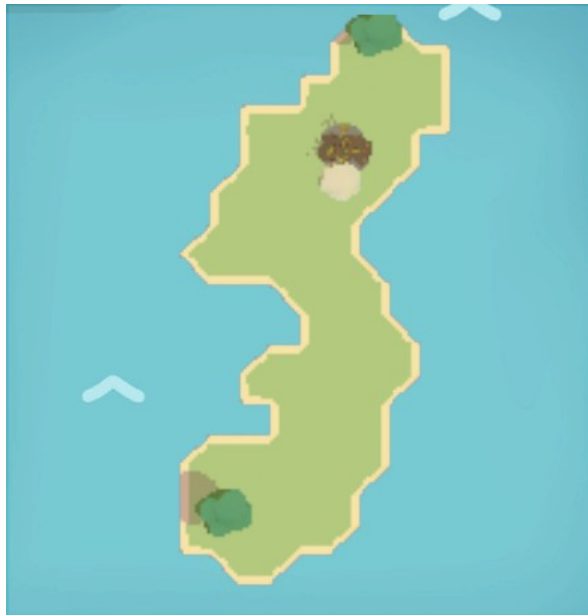
Proly

- Flag capturing 2.5D game

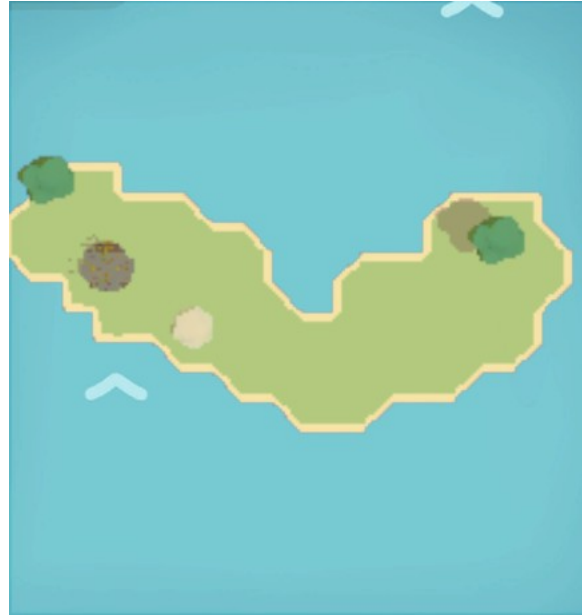


Task

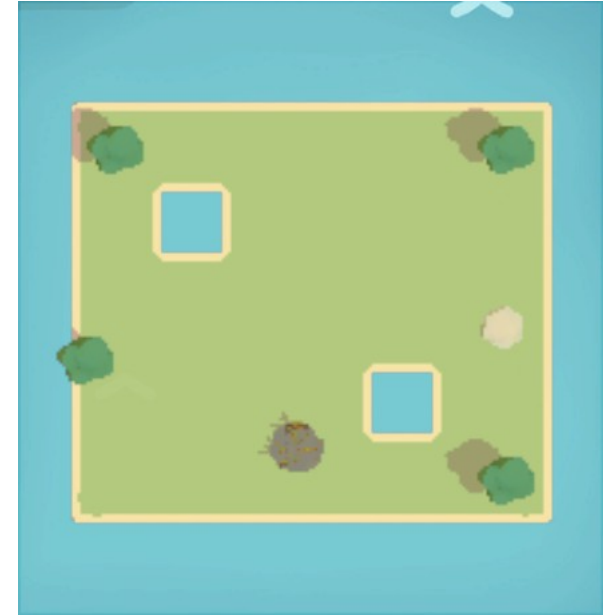
- Design reward function to train an RL agent to play Proly
- We provide 3 public map and 2 private map for grading, 10% each



map 1



map 2



map 3

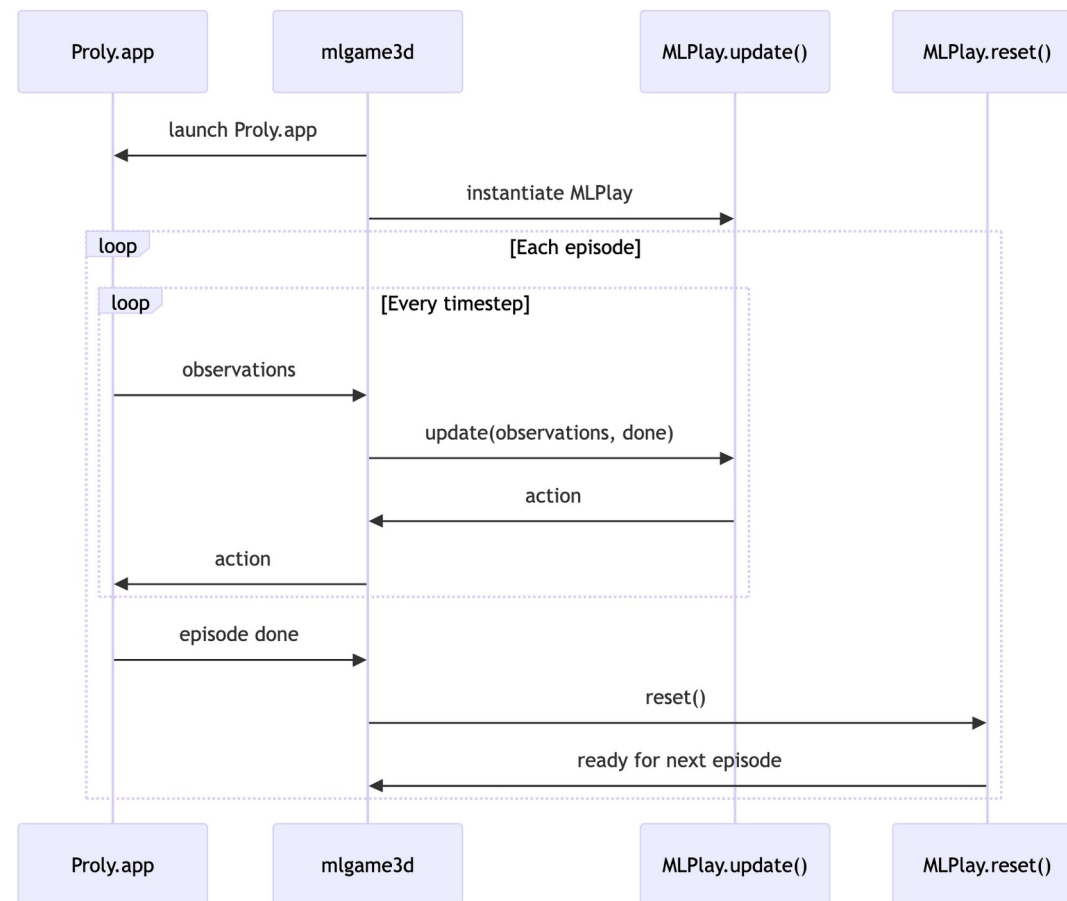
Repo Structure

- HW3-2/
 - **models/**/*.zip**
 - Model checkpoints
 - **model.zip**
 - Latest model checkpoint
 - **rl_play.py (TODO)**
 - Contains MLPlay class to train model with PPO
 - **model_play.py**
 - Contains MLPlay class to play Proly with trained model stored at model.zip
 - **kb_play.py**
 - Contains MLPlay class to play Proly with keyboard, useful for debugging
 - **dummy_env.py**
 - A dummy class used to initialize StableBaselines' PPO policy (**DO NOT MODIFY**)
 - **Proly.app/.exe/.x86_06 (depends on OS)**

Submit **model.zip**, **rl_play.py** and **model_play.py**

MLGame3D

- A bridge between the Unity game and RL algorithm



(rl_play.py) Walkthrough

- **MLPlay**

- **`__init__(self, observation_structure, action_space_info, ...)`**
 - Initialize hyperparameters in `self.config`
 - Initialize PPO policy model via `self._initialize_model`
- **`update(self, observation, done, ...) -> action`**
 - Called at each timestep, receives observation & predicts action with policy via `self._predict_action`
 - Add (observation, action, reward, etc.) to rollout buffer via `self._add_to_rollout_buffer`
- **`reset(self)`**
 - Update policy with rollout datas via `self._update_policy`
 - Clear rollout buffer
- **`_initialize_model(self)`**
 - Load model if `model.zip` exists, initialize otherwise
- **`_save_model(self)`**
 - Save current model checkpoint to `self.model_save_dir` and `self.model_path`
- **`_predict_action(self, obs)`**
- **`_add_to_rollout_buffer(self, obs, action, reward, done, value, log_prob)`**
- **`_update_policy(self)`**

TODO 6 (rl_play.py)

- **RewardManager**

- **update(self, observation)**

- Update reward manager with current observation & cache previous observation in `self.prev_observation`. Called by `MLPlay.update`

- **reset(self)**

- Clean cached observations. Called by `MLPlay.reset`

Implement

- **calculate_reward(self) -> float # required**

- Calculates the total reward signal at each timestep. Called by `MLplay.update` after `RewardManager.update`

- **calculate_flag_capture_reward(self) -> float # optional**

- **Hint:** When a new flag is captured, give a large reward to encourage the agent to move along the correct path.

- **calculate_distance_reward(self) -> float # optional**

- **Hint:** Guide the agent to constantly move closer to the target point.

- **calculate_survival_reward(self) -> float # optional**

- **Hint:** Teach the agent the importance of survival - avoid jumping off the cliff

(model_play.py) Walkthrough

```
class MLPlay:
    def __init__(self, observation_structure, action_space_info, *args, **kwargs):
        self.dummy_env = DummyEnv(observation_structure, action_space_info)

        current_dir = os.path.dirname(os.path.abspath(__file__))
        self.model_path = os.path.join(current_dir, "model.zip")

        if os.path.exists(self.model_path):
            print(f"Loading model from: {self.model_path}")
            self.model = PPO.load(self.model_path)
        else:
            print(f"ERROR: model.zip not found at {self.model_path}")
            self.model = None

    def reset(self):
        pass

    def update(self, observations, done, info, *args, **kwargs):
        observation = observations.get("flattened")

        if observation is None or self.model is None:
            # NOTE DO NOT MODIFY.
            # Sending additional dummy discrete actions that would not be needed for this assignment
            return np.zeros(self.dummy_env.action_space.shape), (0, 0)

        action, _ = self.model.predict(observation, deterministic=True)

        # NOTE DO NOT MODIFY.
        # Sending additional dummy discrete actions that would not be needed for this assignment
        return action, (0, 0)
```

Load pre-trained
weights

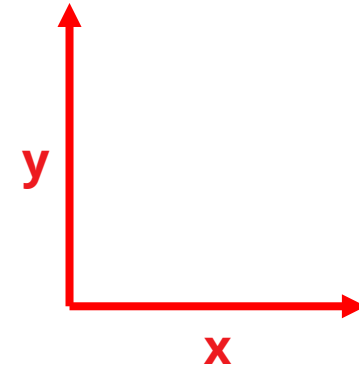
Model inference

Observation Space

- **target_position:** Relative position to the next checkpoint
- **agent_health:** Current health of the agent
- **last_checkpoint_index:** Index of the last checkpoint passed
- **terrain_grid:** 5x5 grid of terrain features around the agent
- **Detailed specs:** <https://github.com/PAIA-Playful-AI-Arena/Proly/#observation-space>

Action Space

- **Continuous actions:** Acceleration (A_x , A_y)
 - Values range from -1.0 to 1.0 for each axis
 - Represents normalized movement direction
 - Controls acceleration rather than direct velocity
- e.g.
 - (0, 1) moves up
 - (-1, 0) moves left



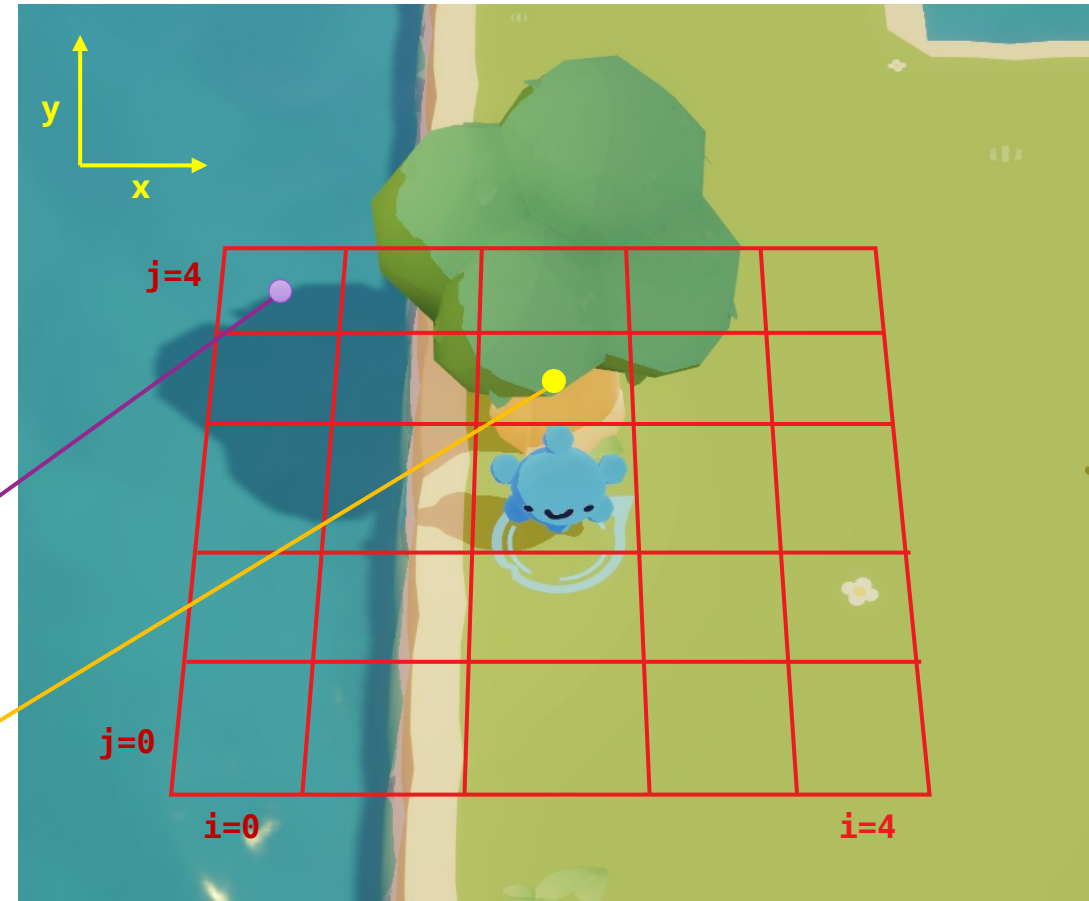
```
def update(self, observations, done, info, keyboard=set(), *args, **kwargs):  
    action = [0, 0]  
    if "up" in keyboard:  
        action[1] = 1  
    elif "down" in keyboard:  
        action[1] = -1  
    elif "left" in keyboard:  
        action[0] = -1  
    elif "right" in keyboard:  
        action[0] = 1  
    return action, (0, 0)
```

terrain_grid

- 5x5 grid of terrain features around the agent
- `self.observation["terrain_grid"][i][j]`:
 - **relative_position**:
 - Grid center's relative position to the player
 - **terrain_type**:
 - **0** : normal
 - **-1** : water
 - **1** : obstacle

```
self.observation["terrain_grid"][0][4] = {  
    "relative_position": (-2, 2),  
    "terrain_type": -1  
}
```

```
self.observation["terrain_grid"][2][3] = {  
    "relative_position": (0, 1),  
    "terrain_type": 1  
}
```



Reward Example

- Check if the *"last_checkpoint_index"* changed between last and current timestep
 - **yes**: captured a flag → give reward
 - **no** → do nothing

```
def calculate_flag_capture_reward(self):  
    if self.prev_observation is None:  
        return 0.0  
  
    prev_checkpoint = self.prev_observation["last_checkpoint_index"]  
    curr_checkpoint = self.observation["last_checkpoint_index"]  
  
    if curr_checkpoint != prev_checkpoint:  
        return 50.0  
    return 0.0
```

(kb_play.py) Walkthrough

- Use arrow key to control the character
- Press space to print observation infos for debugging

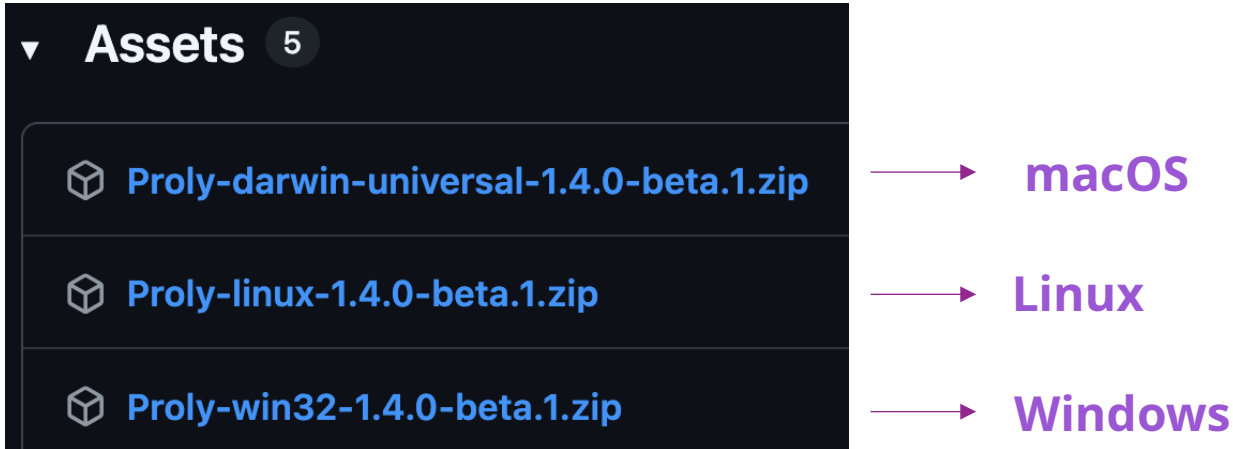
```
def update(self, observations, done, info, keyboard=set(), *args, **kwargs):  
    action = [0, 0]  
    if "up" in keyboard:  
        action[1] = 1  
    elif "down" in keyboard:  
        action[1] = -1  
    elif "left" in keyboard:  
        action[0] = -1  
    elif "right" in keyboard:  
        action[0] = 1  
  
    if "space" in keyboard:  
        # print the observation you wanna check here  
        print(observations["target_position"])  
  
    return action, (0, 0)
```

Setup Requirements

- Python version: 3.12
- Install the following python packages with your package manager of choice
 - `gymnasium==1.2.3`
 - `numpy<2.0.0`
 - `mlgame3d==0.8.0`
 - `stable-baselines3==2.7.1`
 - `tensorboard==2.20.0`
 - `setuptools<81.0` *# important for running tensorboard*
 - `torch==2.10.0`

Install Proly

- **Download page:** <https://github.com/PAIA-Playful-AI-Arena/Proly/releases/tag/1.4.0-beta.1>



▼ **Assets** 5

- Proly-darwin-universal-1.4.0-beta.1.zip → macOS
- Proly-linux-1.4.0-beta.1.zip → Linux
- Proly-win32-1.4.0-beta.1.zip → Windows

- Unzip & put under HW3-2/ directory root
 - **Windows** : Proly.exe
 - **macOS** : Proly.app
 - **Linux** : Proly.x86_64
- To test if the game is working, run:
 - `python -m mlgame3d <Proly executable path>`

Training Command

Game executable path,
make sure to change the extension

of episodes to run

Hide other player, as it is not needed for this assignment (**DON'T MODIFY**)

```
python -m mlgame3d -i rl_play.py -i hidden -i hidden -i hidden -e 10000 \
-gp items 0 -gp audio false -gp map 1 -gp checkpoint 10 -gp max_time 120 Proly.app
```

Disable items(**DONT MODIFY**) & audio

Map to load, can be 1, 2 or 3

of checkpoints

Time limit for each episode in secs

Other useful arguments:

- **-ng**: Disable game rendering
- **-f <int>**: Frame per second
- **-ts <int>**: Time scale factor
- **--seed <int>**: Random seed

Inferencing Command

- Play with trained model from model.zip, we will be grading your model with this same arguments, except for map # and random seed

```
python -m mlgame3d -i model_play.py -i hidden -i hidden -i hidden -e 10000 \  
-gp items 0 -gp audio false -gp map 3 -gp checkpoint 10 -gp max_time 120 Proly.app
```

Visualizing Training Metrics

- Run the following command to launch TensorBoard:

```
tensorboard --logdir tensorboard/
```

- Then visit <http://localhost:6006/> to see various metrics logged during training



Submission

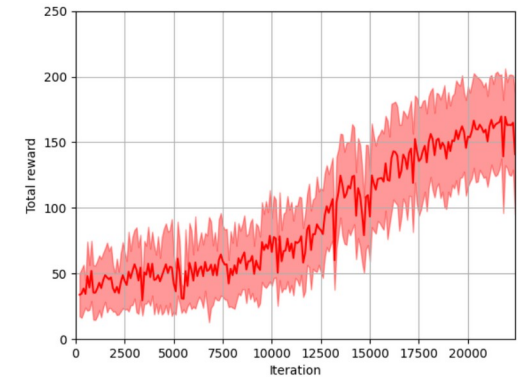
Implementation Rules

- You should only modify the following files
 - **HW3-1/**
 - `model.py`
 - `env_runner.py`
 - `agent.py`
 - `train.py`
 - **HW3-2/**
 - `rl_play.py`
 - `model_play.py`
- For HW3-2, do not implement hard coded gameplay rule in `MLPlay.update()`
- Use of additional .py scripts & packages are prohibited
- Violating rules above will result in a score of zero

HW3-1: Score & Requirement

- **plot.py(10%)**

- Graded based on the plotted graph drawn from `save/return.txt`
- Make sure the overall reward goes up
- Do not copy other's `return.txt`, violation will result in a score of zero for HW3



- **eval.py(30%)**

- Graded based on the evaluation score
- $30 * ES / 120$, where ES is your evaluation score (you will get all 30% if $ES > 120$)

```
Total reward = 258.102730, length = 320
Total reward = 206.884708, length = 325
Total reward = 151.192916, length = 307
Total reward = 169.869626, length = 282
Total reward = 150.867508, length = 306
Total reward = 174.895304, length = 298
Total reward = 170.725990, length = 295
Evaluation Score: 159.6902
```

- **Report(5%)**

- Briefly explain your implementation

Note:

Do not submit `plot.py` and `eval.py`, we'll evaluate based on the original script, so make sure to not touch those during your own testing.

HW3-2: Score & Requirement

- **Agent evaluation(50%)**

- You're agent will be evaluated on 5 different maps(map 1/2/3 and two private maps), each with 10 flags, **1% per flag captured**. Thus a total of 50%
- We'll run each map three times with different random seeds
- Your score for each map will be the highest score across those 3 runs

- **Report(5%)**

- Briefly explain your implementation

Submission Rules

- Submission to NTU COOL should be a zipped file with following structure:

```
HW3_<your_student_id>.zip
├── HW3-1/
│   ├── save/
│   │   ├── model.pt
│   │   └── return.txt
│   ├── agent.py
│   ├── env_runner.py
│   ├── model.py
│   ├── train.py
│   └── report_<your_student_id>.pdf
└── HW3-2/
    ├── model.zip
    ├── model_play.py
    ├── rl_play.py
    └── report_<your_student_id>.pdf
```

- DO NOT include any other folder/files
- Any violation on the rules will result in a deduction of (5 points)/(each violation)

Deadline

- Same as announcement on NTU COOL (2026/4/12 23:59)
- **Hard deadline, no late submission allowed**

Question